

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Колледж программирования и кибербезопасности

УЧЕБНАЯ ДИСЦИПЛИНА/
МЕЖДИСЦИПЛИНАРНЫЙ КУРС
МДК.01.04 Системное программирование

ПЦК Программирования

ВИД УЧЕБНОГО
МАТЕРИАЛА Лекционные материалы

ПРЕПОДАВАТЕЛЬ Иванова Дарья Андреевна

Лекция 3.

Ввод-вывод и работа с файлами

Файл в языке Си рассматривается как неструктурированная последовательность байтов. С этой точки зрения в языке программирования С файлом может быть как собственно файл на жестком диске, так и принтер, дисплей и другие подключаемые устройства ввода-вывода.

Как правило, взаимодействие между приложением и файлами производится посредством обмена блоков байт фиксированной длины (обычно длина представляет степень двойки - 256 или 512 байт).

При чтении из файла данные помещаются в буфер операционной системы, а затем побайтно передаются приложению.

При записи в файл данные накапливаются в буфере, а при заполнении буфера записываются на диск в виде единого блока байт.

Буферы представляют участки памяти, поэтому передача данных между приложением и буфером происходит довольно быстро в отличие от взаимодействия с физическими устройствами типа принтера.

Файл вместе с предоставляемыми средствами буферизации представляет **поток**.

Язык программирования Си содержит необходимый функционал для работы с файлами и устройствами ввода-вывода. Для применения его применения в программе необходимо подключить заголовочный файл **stdio.h**.

Открытие и закрытие потоков

Чтобы работать с потоком, его необходимо открыть. Для открытия потока применяется функция **fopen()**, которая имеет следующий прототип:

```
1  FILE * fopen(имя_файла, режим_открытия);
```

Первый параметр представляет имя открываемого файла, а второй задает режим открытия, от которого зависит, как файл может быть обработан.

Функция возвращает указатель на структуру, которая имеет тип **FILE**, определенный в том же файле stdio.h. Этот указатель идентифицирует поток в программе и через него мы сможем обращаться к открытому файлу.

При открытии поток связывается со структурой

Режимы открытия

Каждый режим задается в виде набора символов. В частности, мы можем использовать следующие режимы:

- **"w"**: текстовый файл открывается для записи. Если файл ранее существовал, то он пересоздается и записывается заново

- **"r"**: текстовый файл открывается для чтения
- **"a"**: текстовый файл открывается для добавления в него новых данных. Если файл существовал ранее, то данные просто добавляются
- **"w+"**: текстовый файл создается для записи/записи. Если файл ранее существовал, то при первой записи после открытия он пересоздается и записывается заново. А при последующих записях после открытия данные добавляются в него без перезаписи.
- **"r+"**: текстовый файл открывается для чтения/записи. Запись допустима в любом месте файла, кроме конца файла, то есть недопустимо увеличение размеров файла.
- **"a+"**: текстовый файл открывается или создается (при его отсутствии) для чтения/записи. В отличие от режима w+ файл при открытии не пересоздается заново, а в отличии от режима r+ можно записывать данные в конец файла
- **"wb"**: бинарный файл открывается для записи
- **"rb"**: бинарный файл открывается для чтения
- **"ab"**: бинарный файл открывается для дозаписи
- **"w+b"**: бинарный файл создается для записи/чтения
- **"r+b"**: бинарный файл открывается для чтения/записи
- **"a+b"**: бинарный файл открывается или создается (при его отсутствии) для чтения/дозаписи

Режимы позволяют разграничить доступ к файлу и открыть его только для чтения или только для записи или совместить оба варианта. Кроме того, на уровне режимов происходит разделение файлов на текстовые и бинарные. И программа будет обрабатывать файлы определенным образом, в зависимости какой режим будет выбран - для текстовых или бинарных файлов. Неправильно заданный режим может привести к некорректной интерпретации файла.

Закрытие файла

После завершения работы с файлом его следует закрыть. Для этого применяется функция **fclose()**:

```
int fclose(указатель_на_поток);
```

Единственный параметр функции представляет ранее полученный при открытии файла указатель на структуру FILE, связанный с файлом.

Функция возвращает число: 0 - в случае успешного выполнения и встроенное значение **EOF** в случае ошибки.

Например, откроем и закроем файл "data.txt", который расположен в одной и той же папке, что и выполняемая программа:

```
#include <stdio.h>

int main(void)
{
    FILE * fp = fopen("data.txt", "w");
    fclose(fp);
    return 0;
}
```

Если файл расположен в каком-то другом месте, то можно указать соответствующий относительный или абсолютный путь. Например, "C:\\data.txt" (если файл `data.txt` расположен в корне диска C).

Обработка ошибок

В процессе открытия или создания файла мы можем столкнуться с рядом ошибок, например, при открытии в режиме чтения не окажется подобного файла, недостаточно памяти и т.д. И в случае возникновения ошибки функция `fopen()` возвращает значение **NULL**. Мы можем обработать возникновение ошибки с помощью проверки результата функции:

```
#include <stdio.h>

int main(void)
{
    FILE * fp= fopen("data28.txt", "r");
    if(fp==NULL)
    {
        perror("Error occured while opening data28.txt");
        return 1;
    }

    fclose(fp);
    return 0;
}
```

Для вывода ошибки на консоль применяется встроенная функция **perror()**. И так как дальнейшие действия в программе в случае ошибки при открытии файла смысла не имеют, то с помощью вызова **return 1;** завершаем работу приложения и возвращаем число, отличное от нуля (обычно ненулевое число рассматривается как код ошибки).

И если при попытке открытия файла по указанному пути его не окажется, то консоль выведет следующую ошибку:

```
Error occured while opening data28.txt: No such file or directory
```

Чтение и запись бинарных файлов

Работа с бинарными файлами производится посредством посимвольного чтения-записи. Поскольку один объект `char` соответствует одному байту, то фактически речь идет о побайтовых записи/чтения.

Запись файла

Для записи символа в файл применяется функция **putc()**, которая имеет следующий прототип:

```
int putc(int c, FILE *stream);
```

В качестве параметров в функцию передаются указатель на файловый поток и символ, который надо записать. Результатом функции является записанный символ.

```
#include <stdio.h>

int main(void)
{
    // записываемая строка
    char message[] = "Hello METANIT.COM!";
    // файл для записи
    char * filename = "data.txt";
    // находим длину записываемой строки
    int n = sizeof(message)/sizeof(message[0]);

    FILE * fp= fopen(filename, "w");
    if(fp)
    {
        // посимвольно записываем в файл
        for(int i=0; i < n; i++)
        {
            putc(message[i], fp);
        }

        fclose(fp);
        printf("File has been written\n");
    }
}
```

Здесь мы пробегаемся по массиву символов message и каждый из них заносим в файл data.txt (который располагается в текущей папке), передавая каждый символ в функцию `putc()`.

Чтение файла

Для чтения одного символа применяется функция **getc()** (функция **fgetc()** аналогична). Она имеет следующий прототип:

```
int getc(FILE *stream);
```

В качестве параметра в функцию передается указатель на файловый поток, а возвращаемым значением является считанный из файла символ, точнее его числовой код.

При чтении из файла и записи в него одного символа указатель текущей позиции в потоке увеличивается, обеспечивая тем самым продвижение по файлу.

Считаем ранее записанный файл `data.txt`:

```
#include <stdio.h>

```

```
int main(void)
{
    // символ для считывания
    char c;
    // файл для чтения
    char * filename = "data.txt";

    FILE * fp= fopen(filename, "r");
    if(fp)
    {
        // считываем посимвольно из файла
        while((c=getc(fp))!= EOF)
        {
            printf("%c", c);
        }
        fclose(fp);
    }
}
```

Здесь считываем по одному символу из файла через вызов `getc()`. Здесь стоит обратить внимание на процесс считывания файла. Когда будет достигнут конец файла, функция getc() возвратит значение **EOF** ("end of file" - конец файла)., которое определено в файле stdio.h. Поэтому мы можем считывать файл, пока не получим значение EOF.

В итоге на консоль будет выведено ранее записанное сообщение.

Чтение и запись структур в файл

Хотя функции **getc()/putc()** позволяют вносить в файл отдельные символы, но фактически мы имеем дело с бинарными файлами. Если мы записываем в файл строку, то в принципе мы даже можем открыть записанный файл любом текстовом редакторе и понять, что там было записано. Но не всегда данные могут представлять строки. И чтобы более наглядно разобраться с работой с бинарными файлами, рассмотрим еще одни пример - с записью-чтением структуры из файла.

```
#include <stdio.h>
#include <stdlib.h>

struct person
{
    char name[16];
    int age;
};

int save(char * filename, struct person *p);
int load(char * filename);

int main(void)
{
    char * filename = "person.dat";
    struct person tom = { "Tom Smith", 21 };
```

```
17
18      save(filename, &tom);
19      load(filename);
20
21      return 0;
22  }
23
24  // запись структуры в файл
25  int save(char * filename, struct person *p)
26  {
27      FILE * fp;
28      char *c;
29      int size = sizeof(struct person); // количество записываемых
30  байтов
31
32      fp = fopen(filename, "wb"); //открываем файл для бинарной
33  записи
34      if (!fp)    // если не удалось открыть файл
35      {
36          printf("Error occured while opening file \n");
37          return 1;
38      }
39      // устанавливаем указатель на начало структуры
40      c = (char *)p;
41      // посимвольно записываем в файл структуру
42      for (int i = 0; i < size; i++)
43      {
44          putc(*c++, fp);
45      }
46      fclose(fp);
47      return 0;
48  }
49
50  // загрузка из файла структуры
51  int load(char * filename)
52  {
53      FILE * fp;
54      char *c;
55      int i; // для считывания одного символа
56      // количество считываемых байтов
57      int size = sizeof(struct person);
58      // выделяем память для считываемой структуры
59      struct person * ptr = malloc(size);
60      fp = fopen(filename, "rb");     // открываем файл для бинарного
61  чтения
62      if (!fp)
63      {
64          printf("Error occured while opening file \n");
65          return 1;
66      }
67
```

```
68      // устанавливаем указатель на начало блока выделенной памяти
69      c = (char *)ptr;
70      // считываем посимвольно из файла
71      while ((i = getc(fp))!=EOF)
72      {
73          *c = i;
74          c++;
75      }
76
77      fclose(fp);
78      // вывод на консоль загруженной структуры
79      printf("%-20s %5d \n", ptr->name, ptr->age);
        free(ptr);
        return 0;
    }
```

В данном случае запись и чтение структуры выделены в отдельные функции: `save()` и `load()` соответственно.

Для записи в функции `save()` через параметр `struct person *p` получаем указатель на сохраняемую структур. Фактически его значением является начальный адрес блока памяти, где располагается структура.

Функция putc записывает отдельный символ в файл, однако нам надо записать структуру. Для этого мы создаем указатель на символ (который по сути представляет один байт) и устанавливаем этот указатель на начало блока памяти, выделенного для структуры.

```
1   c = (char *)p;
```

То есть в данном случае мы получаем адрес в памяти первого байта из блока памяти, которая выделена для структуры. И затем мы можем пройтись по всему этому блоку и получить отдельные байты и занести их в файл:

```
1   for (int i = 0; i < size; i++)
2   {
3       putc(*c++, fp);
4   }
```

И в данном случае нам не важно, какие поля имеет структура, какой она имеет размер. Мы работаем с ней как с набором байт и заносим эти байты в файл. После занесения каждого отдельного байта в файл указатель с в блоке памяти перемещается на один байт вперед.

При чтении файла в функции `load()` используется похожий принцип только в обратную сторону.

Во-первых, для считывания структуры из файла мы выделяем блок динамической памяти для хранения прочитанных данных:

```
1   struct person * ptr = (struct person *) malloc(size);
```

После этого указатель ptr будет указывать на первый адрес блока из 20 байт (наша структура занимает 20 байт = 16 символов и 4 байта для числа `int`).

Затем так как при прочтении мы получаем символы, устанавливаем указатель на первый байт выделенного блока и в цикле считываем данные из файла в этот блок:

```
c = (char *)ptr;
// считываем посимвольно из файла
while ((i = getc(fp))!=EOF)
{
    *c = i;
    c++;
}
```

Здесь стоит обратить внимание на то, что в данном случае на самом деле считываем даже не символ, а числовой код символа в переменную типа int и только потом передаем значение указателю c. Это сделано для корректной обработки окончания файла EOF. Это значение может представлять любое отрицательное число. И если бы мы сохранили отрицательное число (например, возраст пользователя был бы отрицательным), то оно было бы некорректно интерпретировано при чтении как конец файла, и итоговый результа был бы неопределенным. Поэтому более правильно считывать именно числовой код символа в переменную int, а затем числовой код передавать в char.

Запись и чтение массива структур

Выше приведен пример по работе с одной структурой. Но, как правило, при работе с файлами мы оперируем не одной структурой, а каким-то набором структур. Поэтому усложним задачу и сохраним и считаем из файла массив структур:

```
#include <stdio.h>
#include <stdlib.h>

struct person
{
    char name[20];
    int age;
};

int save(char * filename, struct person *st, int n);
int load(char * filename);

int main(void)
{
    char * filename = "people.dat";
    struct person people[] = { {"Tom", 23}, {"Alice", 27}, {"Bob", 31},
{"Kate", 29 }};
    int n = sizeof(people) / sizeof(people[0]);

    save(filename, people, n);
    load(filename);
    return 0;
}

// запись в файл массива структур
int save(char * filename, struct person * st, int n)
```

```
{
    char *c;    // для записи отдельных символов
    // число записываемых байтов
    int size = n * sizeof(struct person);
    FILE * fp = fopen(filename, "wb");
    if (!fp)
    {
        printf("Error occured while opening file\n");
        return -1;
    }
    // записываем количество структур
    c = (char *)&n;
    for (int i = 0; i < sizeof(n); i++)
    {
        putc(*c++, fp);
    }

    // посимвольно записываем в файл все структуры
    c = (char *)st;
    for (int i = 0; i < size; i++)
    {
        putc(*c, fp);
        c++;
    }
    fclose(fp);
    return 0;
}

// загрузка из файла массива структур
int load(char * filename)
{
    char *c;    // для считывания отдельных символов
    int m = sizeof(int);    // сколько надо считать для получения размера
массива
    int n;  // количество структур в массиве

    FILE * fp = fopen(filename, "r");
    if (!fp)
    {
        printf("Error occured while opening file\n");
        return -1;
    }
    // выделяем память для хранения количества данных
    int *ptr_count = malloc(m);
    // считываем количество структур
    c = (char *)ptr_count;
    // пока не считаем m байт, сохраняем байт в выделенный блок для размера
массива
    while (m > 0 && (*c = getc(fp)) != EOF)
    {
        c++;
```

```
78          m--;
79      }
80      //получаем число элементов
81      n = *ptr_count;
82      free(ptr_count);    // освобождаем память
83      // выделяем память для считанного массива структур
84      struct person * ptr = malloc(n * sizeof(struct person));
85      // устанавливаем указатель на блок памяти, выделенный для массива
86  структур
87      c = (char *)ptr;
88      // считываем посимвольно из файла
89      while ((*c= getc(fp))!=EOF)
90      {
91          c++;
92      }
93      // перебор загруженных элементов и вывод на консоль
94      printf("\nFound %d people\n\n", n);
95
96      for (int i = 0; i < n; i++)
97      {
98          printf("%-5d %-10s %5d \n", i + 1, (ptr + i)->name, (ptr + i)->age);
99          // или так
100         // printf("%-5d %-10s %5d \n", i + 1, ptr[i].name, ptr[i].age);
101     }
102
        free(ptr);
        fclose(fp);
        return 0;
    }
```

Данная задача усложнена тем, что нам надо хранить массив структур, количество которых точно может быть неизвестно. Один из вариантов рещения этой проблемы состоит в сохранении некоторой метаинформации о файле в начале файла. В частности, в данном случае в начале файла сохраняется число записанных структур.

Запись во многом аналогична записи одной структуры. Сначала устанавливаем указатель на число `n`, которое представляет количество структур, и все байты этого числа записываем в файл:

```
1  c = (char *)&n;
2  for (int i = 0; i<sizeof(n); i++)
3  {
4      putc(*c++, fp);
5  }
```

Затем подобным образом записываем все байты из массива структур - устанавливаем указатель на первый байт массива структур и записываем `size` байт в файл:

```
1  // посимвольно записываем в файл все структуры
2  c = (char *)st;
3  for (int i = 0; i < size; i++)
4  {
```

```
5       putc(*c, fp);
6       c++;
7   }
```

При чтении нам придется файктически считывать из файла два значения: количество структур и их массив. Поэтому при чтении два раза выделяется память. Вначале для количества элементов:

```
1   int *ptr_count = malloc(m);
```

Затем мы считываем первые 4 байта из файла для получения числа:

```
1   c = (char *)ptr_count;
2   while (m > 0 && (*c = getc(fp)) != EOF)
3   {
4       c++;
5       m--;
6   }
7   //получаем число элементов
8   n = *ptr_count;
```

Затем аналогичные действия проделываем для массива структур.

```
1   struct person * ptr = malloc(n * sizeof(struct person));
2   // устанавливаем указатель на блок памяти, выделенный для массива
3   структур
4   c = (char *)ptr;
5   // считываем посимвольно из файла
6   while ((*c= getc(fp))!=EOF)
7   {
8       c++;
    }
```

И результатом программы должен быть вывод считанных данных:

```
Found 4 people

1       Tom                 23
2       Alice           27
3       Bob                 31
4       Kate            29
```

Чтение и запись в файл с помощью функций fwrite и fread

Встроенные функции **fwrite()** и **fread()** упрощают запись и чтение из файлов сложных данных. Рассмотрим их применение.

fwrite

Функция **fwrite** предназначена для записи данных из массива. Она имеет следующее определение:

```
size_t fwrite( const void *buffer, size_t size, size_t count, FILE
*stream );
```

Эта функция принимает следующие параметры:

- **buffer**: указатель на первый объект из массива, который должен быть записан
- **size**: размер **каждого** объекта
- **count**: количество объектов, которые надо записать
- **stream** указатель на файловый поток для записи

В качестве результата функция возвращает количество записанных объектов. Это количество может быть меньше параметра `count`, если при записи произогла ошибка

Фактически функция записывает каждый объект как массив значений `unsigned char`, вызывая при записи для каждого значения unsigned char функцию **fputc**.

Для простейшего примера запишем строку:

```
#include <stdio.h>

int main() {
    char str[] = "Hello METANIT.COM";
    size_t N = sizeof(str);
    FILE *fp = fopen("data.txt", "w");
    size_t count = fwrite(str, sizeof str[0], N, fp);
    printf("wrote %zu elements out of %zu\n", count,  N);
    fclose(fp);
}
```

Здесь записываем строку str в файл "data.txt". Для получения длины строки используем выражение `sizeof(str)`, а для получения размера одного символа - `sizeof str[0]` (в приниципе здесь можно сразу написать 1 - размер одного символа). И если запись пройдет успешно, то консоль выведет

```
wrote 18 elements out of 18
```

fread

Функция **fread** предназначена для считывания данных из файла в массив. Она имеет следующее определение:

```
size_t fread( const void *buffer, size_t size, size_t count, FILE
*stream );
```

Эта функция принимает следующие параметры:

- **buffer**: указатель на массив, в который надо считать данные

- **size**: размер **каждого** объекта
- **count**: количество объектов, которые надо считать
- **stream** указатель на файловый поток для чтения

В качестве результата функция возвращает количество считанных объектов. Это количество может быть меньше параметра `count`, если при записи произогла ошибка или достигнут конец файла.

Фактически функция считывает данные с помощью функции **fgetc()** в буфер, который рассматривается как массив значений `unsigned char`.

Для простейшего примера считаем строку из вышезаписанного файла:

```
#include <stdio.h>

int main() {
    int N  = 20;
    char str[N];   // определяем буфер достаточной длины

    FILE *fp = fopen("data.txt", "r");
    size_t count = fread(str, sizeof str[0], N, fp);
    printf("read %zu elements out of %d\n", count,  N);
    printf(str);
    fclose(fp);
}
```

Здесь считываем данные в строку str. В итоге консольный вывод будет следующим:

```
read 18 elements out of 20
Hello METANIT.COM
```

Отслеживание ошибок

При записи или чтении могут возникнуть ошибки. Например, возьмем чтение данных. Нам может потребоваться считать определенный размер данных, но вполне возможно, что в файле не окажется такого количества данных, и мы дойдем до конца файла. Конец файла можно отследить с помощью функции **feof()**, в которую передается указатель на файл и которая возвращает 0, если достигнут конец файла. Дополнительно с помощью функции **ferror()**, в которую также передается указатель на файл, можно узнать произошла ли ошибка при работе с файлом. Например, считаем данные из файла:

```
#include <stdio.h>

int main() {
    int N  = 20;
    char str[N];   // определяем буфер достаточной длины

    FILE *fp = fopen("data.txt", "r");
    int count = fread(str, sizeof str[0], N, fp);
    if(count == N)
    {
```

```
11          printf("%s \n", str);
12      }
13      else
14      {
15          if (feof(fp))   // если конец файла
16          {
17              printf("Unexpected end of file\n");
18              printf("Available: %s",str);
19          }
20          else if (ferror(fp))    // если ошибка
21              perror("Error while reading file\n");
22      }
23      fclose(fp);
24  }
```

В данном случае с помощью выражения `feof(fp)` проверяем, достигнут ли конец файла. Причем конец файла сам по себе не означает ошибку. Наличие же ошибки проверяется с помощью выражения `ferror(fp)`

Запись и чтение структур

Запись и чтение строки - не самая сложная задача, которую можно выполнить и другими способами. Но функции fwrite и fread позволяют работать и с другими объектами, даже если они и не представляют массив. Поскольку функции в качестве буфера принимают указатель, но в качестве указателя может выступать адрес любого объекта. Рассмотрим, как записать и считать структуры. Для этого определим следующую программу:

```
1   #include <stdio.h>
2
3   struct person
4   {
5       char name[20];
6       int age;
7   };
8
9   int main() {
10      // запись файла
11      struct person tom = {"Tom", 22}; // структура для записи
12      int size = sizeof(struct person);
13      FILE *fp = fopen("person.bin", "w");
14      // записываем одну структуру
15      size_t count = fwrite(&tom, size, 1, fp);
16      printf("wrote %zu elements out of %d\n", count,  1);
17      fclose(fp);
18
19      // считывание структуры
20      struct person unknown;  // структура для чтения
21      fp = fopen("person.bin", "r");
22      // считываем одну структуру
23      count = fread(&unknown, size, 1, fp);
24      if(count == 1)
```

```
    {
        printf("Name: %s \n", unknown.name);
        printf("Age: %d \n", unknown.age);
    }
    fclose(fp);
}
```

Здесь записываем структуру типа person. Для этого в функцию `fwrite()` передаем адрес структуры:

```
fwrite(&tom, size, 1, fp);
```

После записи считываем данные в структуру unknown, для чего передаем в функцию `fread()` адрес этой структуры:

```
fread(&unknown, size, 1, fp);
```

Запись-чтение массива структур

Подобным образом можно записывать и считывать массив структур. Для этого определим следующую программу:

```
#include <stdio.h>

struct person
{
    char name[20];
    int age;
};

int main() {
    char * filename = "people.bin";

    // массив для записи
    struct person people[] = { {"Tom", 23}, {"Alice", 27}, {"Bob", 31},
{"Kate", 29 }};
    int size = sizeof(people[0]);              // размер всего массива
    int count = sizeof(people)  / size;         // количество структур

    // запись файла
    FILE *fp = fopen(filename, "w");
    // записываем массив структур
    size_t written = fwrite(people, size, count, fp);
    printf("wrote %zu elements out of %d\n", written, count);
    fclose(fp);

    // считывание файла
    struct person users[count];  // массив для чтения структур
    fp = fopen(filename, "r");
    size_t read = fread(users, size, count, fp);
    printf("read %zu elements\n", read);
```

```
30
31      if(read > 0)
32      {
33          for(int i = 0; i < count; i++)
34          {
35              printf("Name: %s \t Age: %d\n", users[i].name, users[i].age);
36          }
37      }
38      fclose(fp);
    }
```

В данном случае записываем в файл массив структур people, а считываем - в массив users. Консольный вывод программы:

```
wrote 4 elements out of 4
read 4 elements
Name: Tom        Age: 23
Name: Alice      Age: 27
Name: Bob        Age: 31
Name: Kate       Age: 29
```

Считывание по одной структуре

В примерах выше разом считывались те данные, которые ранее были записаны. Но допустим, нам нужно вывести на консоль не всех пользователей, а каких-то определенных. Например, тех, которые имеют определенное имя или возраст. Мы опять же могли бы считать весь массив и затем пройтись по нему в цикле, находя в нем нужные данные. Но также мы можем считывать непосредственно из файла по одному объекту и проверять его. Например, определим следующую программу:

```
1   #include <stdio.h>
2
3   struct person
4   {
5       char name[20];
6       int age;
7   };
8
9   void save(char*);
10  void read_by_age(char*, int);
11
12  int main() {
13      char * filename = "people.bin";
14      save(filename);
15      read_by_age(filename, 22);   // находим пользователей, которым 22 года
16  }
17
18  // записываем данные
19  void save(char* filename)
```

```
{
    struct person people[] = { {"Tom", 22}, {"Bob", 33}, {"Kate", 33 },
{"Alice", 22}};
    int size = sizeof(people[0]);              // размер всего массива
    int count = sizeof(people)  / size;         // количество структур

    FILE *fp = fopen(filename, "w");
    size_t written = fwrite(people, size, count, fp);
    printf("wrote %zu elements out of %d\n", written, count);
    fclose(fp);
}
// выводим данные на консоль для возраста age
void read_by_age(char* filename, int age)
{
    struct person p;  // структур для считывания файла
    FILE* fp = fopen(filename, "r");
    while(fread(&p, sizeof(p), 1, fp) == 1)
    {
        if(p.age == age)    // если пользователь имеет определенный возраст
        {
            printf("Name: %s \t Age: %d \n", p.name, p.age);
        }
    }
    fclose(fp);
}
```

Здесь запись и чтение разбиты на отдельные функции. Но если в функции `load()` записываем в файл разом весь массив структур, то в функции `read_by_age()` считываем по одной структуре в цикле, пока в файле есть данные. И если поле age структуры соответствует параметру age, то выводим эту структуру на консоль.

Чтение и запись текстовых файлов

При работе с текстовыми файлами гораздо проще работать с данными не как с отдельными символами, а как со строками с помощью функций **fgets()** и **fputs()**.

Запись текстового файла

Функция **fputs()** записывает в файл строку, то есть набор символов, который завершается символом '\0'. Она имеет следующий прототип:

```
int fputs(const char *s, FILE *stream);
```

Первый параметр функции - записываемая строка, а второй - указатель на файловый поток. В качестве результата функция возвращает неотрицательное целое число. При ошибках в процессе записи возвращается значение **EOF**.

При записи строки нулевой символ '\0' в файл не записывается.

Например, запишем строку в файл:

```
#include <stdio.h>

int main(void)
{
    // строка для записи
    char * message = "Hello METANIT.COM!\nAn apple a day keeps the
doctor away";
    // файл для записи
    char * filename = "data.txt";
    // запись в файл
    FILE *fp = fopen(filename, "w");
    if(fp)
    {
        // записываем строку
        fputs(message, fp);
        fclose(fp);
        printf("File has been written\n");
    }
}
```

Запись довольно проста: открываем файл на запись и с помощью вызова `fputs(message, fp)` записываем в файл искомую строку.

Чтение текстового файла

Функция **fgets()** считывает из файла одну строку и имеет следующий прототип:

```
char * fgets(char *s, int n, FILE *sream);
```

Первый параметр `char *s` представляет строку, в которую производится считывание. Второй параметр - число `n` указывает, сколько символов необходимо считывать. И третий параметр представляет файловый поток, из которого производится считывание.

При вызове функция считывает из файла не более n-1 символов. Функция прекращает чтение, когда прочтет n-1 символов или встретит символ переноса строки \n. Все считанные символы записываются в строку s, в том числе символ \n. И также конец каждой строки дописывается нулевой символ '\0'.

При успешном завершении функция возвращает указатель `s`, а при ошибке или достижении конца файла возвращается значение **NULL**.

Считаем данные из выше записанного файла "data.txt":

```
#include <stdio.h>

int main(void)
{
    // файл чтения
    char * filename = "data.txt";
    // буфер для считавания данных из файла
    char buffer[256];
    // чтение из файла
```

```
    FILE *fp = fopen(filename, "r");
    if(fp)
    {
        // пока не дойдем до конца, считываем по 256 байт
        while((fgets(buffer, 256, fp))!=NULL)
        {
            printf("%s", buffer);
        }
        fclose(fp);
    }
}
```

Здесь открываем файл на чтение и в цикле считываем из файла по 256 символов и выводим их на консоль с помощью вызова `fgets(cc, 256, fp)`. Когда данные в файле закончатся, функция возвратит NULL, и произойдет выход из цикла.

Копирование файлов

Другой пример работы с текстовыми файлами - копирование содержимого из одного файла в другой:

```
#include <stdio.h>

int main(void)
{
    char * filename1 = "data1.txt";
    char * filename2 = "data2.txt";
    char buffer[256];
    FILE *f1 = fopen(filename1, "r");   // файл на чтение
    FILE *f2 = fopen(filename2, "w");   // файл на запись
    if(!f1 || !f2)
    {
        printf("Error occured while opening file\n");
    }
    else
    {
        // пока не дойдем до конца, считываем по 256 байт из файла
f1
        while((fgets(buffer, 256, f1))!=NULL)
        {
            // записываем строку в файл f2
            fputs(buffer, f2);
            printf("%s", buffer);
        }
    }

    fclose(f1);
    fclose(f2);

    return 0;
}
```

Форматируемый ввод-вывод

Еще один способ чтения и записи информации в файлах представляют функции **fprintf()** и **fscanf()**. Они позволяют отформатировать записываемые и считываемые данные соответственно.

Форматированный вывод

Функция **fprintf()** имеет следующий синтаксис:

```
1   int fprintf(указатель_на_поток, строка_форматирования,
    список_аргументов);
```

Эта функция записывает данные, передаваемые в качестве третьего параметра, в файл с учетом форматирования, определяемого строкой форматирования. Первый параметр представляет указатель на файловый поток, в который производится запись.

Строка форматирования и передача в нее значений определяются также, как и в случае с функцией printf()

Например, запишем в файл массив структур:

```
1   #include <stdio.h>
2
3   struct person
4   {
5       char name[20];
6       int age;
7   };
8   int main(void)
9   {
10      char * filename = "users.dat";
11      struct person people[] = { {"Tom", 23}, {"Alice", 27}, {"Bob", 31},
12  {"Kate", 29} };
13      // количество записываемых структур
14      int n = sizeof(people)/sizeof(people[0]);
15      FILE *fp = fopen(filename, "w");
16      if (!fp)
17      {
18          printf("Error occured while opening file\n");
19          return 1;
20      }
21
22      for(int i=0; i < n; i++)
23      {
24          fprintf(fp, "%s  %d\n", people[i].name, people[i].age);
25      }
26      fclose(fp);
```

```
27      printf("File has been written\n");
28      return 0;
    }
```

Каждая структура имеет два поля - строковое поле name и числовое поле age. Поэтому строка форматирования принимает соответствующие спецификаторы: `"%s %d\n"`.

После записи мы получим текстовый файл users.dat, который сможем просмотреть в любом текстовом редакторе.

Форматированный ввод

Для чтения данных в определенном формате применяется функция **fscanf()**, которая имеет следующий синтаксис:

```
1   int fscanf(указатель_на_поток, строка_форматирования,
    список_аргументов);
```

По факту эта функция похожа на функцию scanf(), которая используется для ввода с консоли, только теперь вод осуществляется из файла, указатель на который передается в качестве первого параметра.

Функция возвращает числовое значение. Если достигнут конец файла, то функция возвращает значение EOF.

Например, считаем и выведем на консоль ранее записанные в файл users.dat данные:

```
1   #include <stdio.h>
2
3   int main(void)
4   {
5       char * filename = "users.dat";
6       char name[20];  // для считывания имени
7       int age;        // для считывания возраста
8       FILE *fp = fopen(filename, "r");
9       if (!fp)
10      {
11          printf("Error occured while opening file\n");
12          return 1;
13      }
14
15      while((fscanf(fp, "%20s  %d\n", name, &age))!=EOF)
16      {
17          printf("%s  %d\n", name, age);
18      }
19      fclose(fp);
20      return 0;
21  }
```

Для считывания данных здесь определены вспомогательные переменные name и age. И при вызове функции fscanf в качестве списка аргументов передаются адреса этих

переменных. Кроме того, надо отметить, что строка форматирования совпадает со строкой, которая передавалась в предыдущем примере при записи в файл: `"%s %d\n"`.

В итоге пока при считывании не будет достигнут конец файла и функция не вернет значение EOF, она будет считывать данные в переменные name и age, которые затем будут выводиться на консоль. В итоге мы получим следующий консольный вывод:

```
Tom             23
Alice   27
Bob             31
Kate    29
```

Позиционирование в файле

Операция чтения-записи всегда производится с текущей позиции в потоке. При открытии потока в режимах **r** и **w** указатель текущей позиции устанавливается на начальный байт потока. При открытии в режиме **a** указатель устанавливается на конец файла сразу за конечным байтом. И при выполнении операции чтения-записи указатель в потоке перемещается на новую позиции в соответствии с числом прочитанных или записанных байтов.

Однако вполне возможно, что нам потребуется считывать или записывать с какой-то определенной позиции в файле. Например, в айдиофайле в формате wav собственно звуковые данные расположены, начиная с 44 байта. И если, к примеру, мы хотим распознать звук из файла, что-то с ним сделать, то нам при считывании данных надо переместить указатель на соответствующую позицию.

В языке Си для управления позицией указателя в потоке применяется функция **fseek()**, которая имеет следующий синтаксис:

```
int fseek(указатель_на_поток, смещение, начало_отсчета);
```

Второй параметр этой функции - *смещение* представляет числовое значение типа **long**, которое указывает, на какое количество байт надо переместить указатель. Это значение может быть отрицательным, если необходимо в потоке вернуться назад на некоторое количество байт.

Третий параметр - *начало_отсчета* задает начальную позицию, относительно которой идет смещение. В качестве этого параметра мы можем использовать одну из встроенных констант, определенных в файле **stdio.h**:

- **SEEK_SET**: имеет значение 0 и представляет начало файла
- **SEEK_CUR**: имеет значение 1 и представляет текущую позицию в потоке
- **SEEK_END**: имеет значение 2 и представляет конец файла

Если перемещение указателя было успешно выполнено, то функция **fseek()** возвращает 0, иначе она возвращает ненулевое значение.

Применим функцию fseek в программе:

```
1   #include <stdio.h>
2
3   void load(char *, int);
4   void save(char *);
5   int main(void)
6   {
7       // файл для записи и чтения
8       char * filename = "data.txt";
9       // позиция, с которой начинается считывание
10      int position = 6;
11      save(filename);
12      load(filename, position);
13
14      return 0;
15  }
16  void load(char * filename, int position)
17  {
18      // буфер для считавания данных из файла
19      char buffer[256];
20      // чтение из файла
21      FILE *fp = fopen(filename, "r");
22      if(!fp)
23      {
24          printf("Error occured while opening file\n");
25          return;
26      }
27      // перемещаем указатель в файле на позицию position
28      fseek(fp, position, SEEK_SET);
29      // пока не дойдем до конца, считываем по 256 байт
30      while((fgets(buffer, 256, fp)))
31      {
```

```
32          printf("%s", buffer);
33      }
34
35      fclose(fp);
36  }
37  void save(char * filename)
38  {
39      // строка для записи
40      char * message = "Hello METANIT.COM!";
41      // запись в файл
42      FILE *fp = fopen(filename, "w");
43      if(!fp)
44      {
45          printf("Error occured while opening file\n");
46          return;
47      }
48      // записываем строку
49      fputs(message, fp);
50
51      fclose(fp);
52      printf("File has been written\n");
53  }
```

Здесь функция `save()` записывает в файл строку "Hello METANIT.COM!". Далее функция `load()` считывет данные из этой строки. Но считывает не сначала, а с позиции, которая передается через параметр `position`:

```
1   // перемещаем указатель в файле на позицию position
2   fseek(fp, position, SEEK_SET);
```

Поскольку третий аргумент равен `SEEK_SET`, то указатель файла смещается к байту с индексом position. Соответственно далее функция `fgets()` будет считывать данные не с самого начала файла, а с позиции position:

```
1   while((fgets(buffer, 256, fp)))
```

Поскольку в данном случае в качестве позиции передается число 6, то в тексте файла будут пропущены первые 6 символов, и будет считана подстрока "METANIT.COM!"

ftell

Кроме функции **fseek()** мы можем использовать для управления позицией указателя еще пару функций:

- **long ftell(FILE *)**: получает текущую позицию указателя
- **void rewind(FILE *)**: указатель устанавливается на начало потока

Например, мы можем применять функцию **ftell** для вычисления длины файла в байтах:

```
#include <stdio.h>

int main(void){

    FILE* fp = fopen("test.txt", "r");
    if(!fp) // если не удалось открыть файл
    {
        printf("Error while opening file\n");
        return -1;
    }
    // если удалось, открыть файл получаем его длину
    fseek(fp, 0, SEEK_END);  // устанавливаем указатель на конец файл
    long size = ftell(fp);   // получаем значение указателя относительно начала
    fclose(fp);     // закрываем файл

    printf("File size: %ld bytes\n", size);
}
```

В данном случае находим длину файла "text.txt", который располагается в папке программы. Само находждение длины разбивается на два этапа. Сначала помещаем указатель в файле на конец с помощью функции `fseek()`:

```
fseek(fp, 0, SEEK_END);
```

Затем с помощью функции `ftell()` вычисляем положение указателя относительно начала файла - фактически получаем размер файла в байтах

```
long size = ftell(fp);
```

Считывание определенной структуры из файла

Рассмотрим более сложный пример:

```
 #include <stdio.h>
```

```
2   #include <stdlib.h>
3
4   struct person
5   {
6       char name[20];
7       int age;
8   };
9
10  int save(char * filename, struct person *st, int n);
11  int load(char * filename);
12
13  int main(void)
14  {
15      char * filename = "people.dat";
16      struct person people[] = { {"Tom", 23}, {"Alice", 27}, {"Bob", 31}, {"Kate"
17      int n = sizeof(people) / sizeof(people[0]);
18
19      save(filename, people, n);
20      load(filename);
21
22      return 0;
23  }
24
25  // запись в файл массива структур
26  int save(char * filename, struct person * st, int n)
27  {
28      char *c;    // указатель для посимвольной записи данных
29      // число записываемых байтов
30      int size = n * sizeof(struct person);
31
32      FILE * fp = fopen(filename, "wb");
33      if (!fp)
34      {
35          printf("Error occured while opening file\n");
36          return 1;
37      }
38      // записываем количество структур
39      c = (char *)&n;
40      for (int i = 0; i < sizeof(n); i++)
41      {
42          putc(*c++, fp);
```

```
43          }
44
45          // посимвольно записываем в файл все структуры
46          c = (char *)st;
47          for (int i = 0; i < size; i++)
48          {
49              putc(*c, fp);
50              c++;
51          }
52          fclose(fp);
53          return 0;
54      }
55
56      // загрузка из файла массива структур
57      int load(char * filename)
58      {
59          char *c;    // указатель на считывания очередного символа
60          int n = sizeof(struct person);  // сколько байт надо считать для структуры
61          int index;  // номер структуры из файла:
62          printf("Enter user number: ");  // ввод номера структуры
63          scanf("%d", &index);
64
65          FILE * fp = fopen(filename, "rb"); // открываем файл на чтение
66          if (!fp)
67          {
68              printf("Error occured while opening file\n");
69              return 1;
70          }
71
72          // выделяем память для количества структур
73          int *ptr_count = malloc(sizeof(int));
74          // считываем количество структур
75          c = (char *)ptr_count;
76          int m = sizeof(int);    // сколько надо считать структур
77          // пока не считаем m байт
78          while (m > 0 && (*c = getc(fp))!=EOF)  // сохраняем байт в выделенный блок
79          {
80              c++;
81              m--;
82          }
83          //получаем число элементов
```

```
84          int count = *ptr_count;
85          free(ptr_count);    // освобождаем память
86          // если номер запрощенной структуры меньше кол-ва структур
87          if(index > count)
88          {
89              printf("User number out of range\n");
90              fclose(fp);
91              return 1;
92          }
93
94          // получаем, на сколько байтов надо перемотать указатель относительно начал
95          int pos = (index-1) * n + 4;
96          // перемещаем указатель на нужную позицию
97          fseek(fp, pos, SEEK_SET);
98
99          // выделяем память для считываемой структуры
100         struct person * ptr = malloc(sizeof(struct person));
101
102         // устанавливаем указатель на начало блока памяти
103         c = (char *)ptr;
104         // после записи считываем посимвольно из файла
105         while(n > 0 && (*c=getc(fp))!=EOF)
106         {
107             c++;
108             n--;
109         }
110         // вывод считанных данных на консоль
111         printf("%-10s %5d \n", ptr->name, ptr->age);
112         free(ptr);
113         fclose(fp);
114         return 0;
115     }
```

С помощью функции save в файл сохраняется массив структур. Затем в функции load считываем одну из структур по введенному номеру.

Для поиска нужной структуры вычисляем позицию:

```
1   int pos = (index-1) * n + 4;
```

Исходя из записи мы знаем, что первые 4 байта в файле отводятся под хранение количества структур, соответственно при чтении нам надо пропустить эти 4 байта.

Например, мы хотим получить первую структуру. По формуле получаем (index-1) * n + 4 = 4. То есть первая структура будет располагаться после 4-го байта. Аналогично вторая структура будет располагаться после n+4 байт, где n - это размер структуры.

Получив позицию, передаем ее в функцию **fseek()**, перемещаемся в потоке и считываем после этого n байт.

```
1    // перемещаем указатель на нужную позицию
2    fseek(fp, pos, SEEK_SET);
```

При последующей операции чтения с помощью функции `getc()`:

```
1    while(n > 0 && (*c=getc(fp))!=EOF)
```

эта функция будет считывать данные с позиции `pos`, по которой располагается структура с номером index.

Результат работы программы:

```
Input user number: 2
Alice                  27
```

Стоит отметить, что для большей краткости здесь убрана проверка на наличие структуры по введенному номеру, так как пользователь может ввести число, которое превышает количество

Обновление структуры в файле

Мы можем использовать указатель в потоке не только для чтения, но и для записи данных. Например:

```
1    #include <stdio.h>
2
3    struct person
4    {
5        unsigned id;
6        char name[10];
7        int age;
8    };
```

```
9
10 void save(char*);
11 void update_age(char*, int, int);
12
13 int main() {
14     char * filename = "people.bin";
15     save(filename);
16     update_age(filename, 2, 33); // в структуре с id=2 устанавл. age=33
17 }
18 void save(char* filename)
19 {
20     struct person people[] = { {1, "Tom", 38}, {2, "Sam", 25},
21 {3, "Bob", 42}};
22     int size = sizeof(people[0]);              // размер всего массива
23     int count = sizeof(people)  / size;         // количество структур
24
25     // запись файла
26     FILE *fp = fopen(filename, "w");
27     // записываем массив структур
28     fwrite(people, size, count, fp);
29     fclose(fp);
30     printf("%d people saved\n", count);
31 }
32
33 void update_age(char* filename, int id, int age)
34 {
35     // считывание файла, пока не найдем структуру с определенным id
36     struct person p;  // структура для чтения
37     int size = sizeof(p);
38     FILE* fp = fopen(filename, "r+");
39     // "r+" - открываем файл для изменения
40     // считываем данные в структуру
41     while(fread(&p, sizeof(p), 1, fp)==1)
42     {
43         // если нашли структуру с нужным id
44         if(p.id == id)
45         {
46             p.age = age;                    // изменяем возраст
47             fseek(fp, -1*size, SEEK_CUR);
48      // перемещаем в потоке на один объект назад
49             fwrite(&p, size, 1, fp);
```

```
50          // записываем обновленную структуру
51                  break;                          // выходим из цикла
52              }
53          }
54          rewind(fp);     // перематываем файл назад
55
56          // считываем по одной структуре и проверяем изменения
57          while(fread(&p, sizeof(p), 1, fp) == 1)
58          {
                printf("Id: %d \t Name: %s \t Age: %d \n", p.id, p.name, p.age);
            }
            fclose(fp);
    }
```

В функции `update_age()` обновляем возраст пользователя по определенному id. Для этого открываем файл с флагом "r+", то есть для чтения с изменением.

```
1   FILE* fp = fopen(filename, "r+");
```

Затем считываем по одной структуре и проверяем id. Если id структуры равен запрошенному id, то изменяем возраст и перезаписываем структуру в файле:

```
1   if(p.id == id)
2   {
3       p.age = age;                    // изменяем возраст
4       fseek(fp, -1*size, SEEK_CUR);   // перемещаем в потоке на один объект назад
5       fwrite(&p, size, 1, fp);      // записываем обновленную структуру
6       break;                        // выходим из цикла
7   }
```

Для перезаписи перемещаемся в файле на начало структуры (`fseek(fp, -1*size, SEEK_CUR)`). Поскольку, когда мы считали структуру, указатель в файле указывает на следующую за ней структуру.

Поскольку id у структур уникальны, соответственно нам надо изменить только один объект, поэтому после изменения объекта выходим из цикла.

Чтобы до закрытия файла вывести в этой же функции все структуры из файла, перемещаем указатель в файле на начало с помощью функции `rewind()`

```
1    rewind(fp);     // перематываем файл назад
```

Таким образом, структура с id=2, где изначально age был равен 27, изменит это значение age на 33. Консольный вывод программы:

```
3 people saved
Id: 1    Name: Tom       Age: 38
Id: 2    Name: Sam       Age: 33
Id: 3    Name: Bob       Age: 42
```